Maina — GitHub Issues Doc

Copy-paste ready. Each issue is self-contained with title, labels, problem, acceptance criteria, dependencies, and size. Grouped by milestone, sequenced across 6 weeks with pre-sprint ADRs.

Labels legend: `epic` , `adr` , `dx` , `onboarding` , `constitution` , `wiki` , `verify` , `docs` , `pr-comment` , `moat` , `observability` , `cloud` , `oss` , `kill` , `marketing` , `p0` , `p1` , `p2` , `size:XS/S/M/L` .

Package reference: npm package is `@mainahq/cli` . Primary install is `curl -fsSL https://api.mainahq.com/install | bash` . Alternates: `bun add -g @mainahq/cli` , `bunx @mainahq/cli` . Binary after install is `maina` .

📋 Milestone 0 — Pre-sprint ADRs

These unblock or gate work downstream. A few hours of decision-writing each. File and resolve before Week 1 starts.

#1 ADR — Report storage backend (R2)

Labels: `adr` , `infra` , `pr-comment` , `p0` , `size:XS` **Blocks:** #20

Document the existing Cloudflare R2 choice for proof artifacts so team and future contributors have a written record. No infra spike — this formalizes what's running.

Acceptance.

- `docs/decisions/0001-report-storage-backend.md` written
- Covers: cost model at 10k runs/month and 100k runs/month, bucket layout for `r/<run-id>/` , signed upload flow from CI, retention policy (90 days default, extensible per plan), GDPR delete path, CDN cache strategy, disaster recovery
- Documents why R2 over S3 / Supabase Storage (zero egress, S3-compatible, cheaper at scale)
- Linked from #20 and any Cloud Verification docs

#2 ADR — Experiment gate criteria for Stagehand and Orama

Labels: `adr`, `experiment`, `p1`, `size:XS` **Blocks:** #44, #45

Written go/no-go criteria *before* the spikes run, or they drift into "we invested time so let's ship it."

Acceptance.

- `docs/decisions/0002-stagehand-gate.md`: ship only if pass rate ≥95% across 50 runs AND token cost per verify ≤$0.05 AND cache hit rate ≥80% after first run
- `docs/decisions/0003-orama-gate.md`: ship only if hybrid beats lexical by ≥15% NDCG@10 AND index size ≤2× lexical AND p95 query latency ≤50ms
- Both written before Week 5 spikes start
- Kill is the default if criteria aren't met

#3 ADR — CLI and MCP are co-equal first-class surfaces

Labels: `adr`, `dx`, `p0`, `size:XS`

Maina serves two user shapes: agent-native (Claude Code / Cursor / Windsurf via MCP) and terminal-native (CI, pre-commit, scripting via CLI). Don't let feature work accidentally privilege one.

Acceptance.

- `docs/decisions/0004-cli-mcp-coequal.md` written
- Design rule: every feature reachable via both `maina <cmd>` and an equivalent MCP tool
- PR-review checklist adds: "Does this feature work equally well from CLI and MCP?"
- mainahq.com shows both paths above the fold (curl command + MCP badges)

#4 ADR — Error reporting backend (Sentry / GlitchTip)

Labels: `adr`, `observability`, `p0`, `size:XS` **Blocks:** #34, #35

Pick and document the error-reporting backend.

Option	Pros	Cons

Sentry (managed)	Industry default, great DX, free tier (5k events/mo), polyglot SDKs	SaaS dep, paid beyond free tier
Sentry (self-hosted)	Full control, OSS, no SaaS dep	Ops burden, infra cost
GlitchTip	Sentry-SDK-compatible, fully free, lightweight	Smaller ecosystem

Acceptance.

- `docs/decisions/0005-error-backend.md` written
- Cost model at 10k and 100k events/month
- Recommendation: Sentry managed for Cloud, GlitchTip self-hosted for OSS (both speak the Sentry SDK protocol — one SDK in code, different DSN per build, zero lock-in)

🗺️ Milestone 1 — Week 1: Onboarding narrative

#5 Epic — Rebuild onboarding narrative for first-time users

Labels: `epic`, `dx`, `onboarding`, `docs`, `p0` **Size:** M

Problem. The install path is excellent (curl script + auto-detect + `maina init --install` + `maina doctor`). The tedium is in the `/getting-started` page itself: 9 sections on one scroll — Quick Start (5 sub-steps) + Supported Languages + Zero-Friction Layers (L0–L3) + "What happens during maina commit" (6-step breakdown) + MCP + CI + Cloud Sync + Cloud Verification. First-time users bounce before running their first `maina commit`.

Goal. Time from landing on `/quickstart` to successful first `maina commit` under 60 seconds. Full-setup info preserved behind one more click.

Acceptance.

- `/quickstart` page exists, ≤200 words total, 3 commands max
- `/getting-started` renamed to `/full-setup` or restructured as reference-only
- Hero CTA points to `/quickstart`
- MCP install badges on hero next to curl command
- Docs numbers (tool count, command count, MCP tool count) consistent across site
- Measurable: time-on-page for quickstart drops, click-through to Commands/MCP rises

Child issues: #6, #7, #8, #9, #10

#6 Split `/getting-started` into `/quickstart` + `/full-setup`

Labels: `onboarding`, `docs`, `p0`, `size:S` **Parent:** #5

`/quickstart` contains exactly this, nothing more:

1. `curl -fsSL https://api.mainahq.com/install | bash`
2. `maina init --install`
3. `maina commit`

Plus a single "Next steps" block linking to MCP setup, CI integration, and full reference.

Defer to `/full-setup`: Zero-Friction Layers table, "What happens during maina commit" 6-step, CI integration, Cloud Sync, Cloud Verification, Supported Languages table, configure instructions.

Acceptance.

- `/quickstart` ≤200 words
- Zero prose block over 2 sentences
- Copy-button on each command
- Works end-to-end on a clean machine in under 60 seconds
- Sidebar "Start Here" section puts Quickstart above Commands

---

#7 MCP install badges on mainahq.com hero

Labels: `onboarding`, `marketing`, `p0`, `size:S` **Parent:** #5

Three badges next to the curl command:

- **Add to Claude Code** → runs `claude mcp add-json maina '{"command":"maina","args":["--mcp"]}'`
- **Add to Cursor** → Cursor deeplink `cursor://anysphere.cursor-deeplink/mcp/install?name=maina&config=<base64>`
- **Add to Windsurf** → copies `~/.codeium/windsurf/mcp_config.json` snippet to clipboard with toast

Acceptance.

- Each badge works end-to-end on a clean machine
- Snippets reference the `@mainahq/cli` -installed `maina` binary
- Badges render above the fold on mobile (≤375px width)
- Base64 encoder for Cursor deeplink verified against Cursor docs
- Unit test or snapshot per generator

#8 Hero polish pass

Labels: `onboarding` , `marketing` , `p1` , `size:XS` **Parent:** #5

Minor tuning, not a rewrite. Keep the 41% hook, keep the asciinema, keep the comparison table. Change only:

- Move "19+ tools · 12 AI tools · 0 config" below the fold (cognitive clutter)
- Ensure asciinema autoplays muted, loops, ≤200KB
- Add MCP badges (pairs with #7)
- CTA button points to `/quickstart` not `/getting-started`

Acceptance.

- Lighthouse perf ≥95
- LCP ≤1.5s on 4G
- No layout shift on badge render

#9 Progressive MCP tool disclosure

Labels: `onboarding` , `mcp` , `p1` , `size:M` **Parent:** #5

Expose only 2–3 high-value MCP tools at session start (`verify` , `getContext` , `reviewCode`). Add a meta-tool `list_tools` the agent calls to fetch the other 7. Saves ~20k tokens of client context.

Acceptance.

- Default MCP handshake returns ≤3 tools

- `list_tools` returns all 10 with short descriptions
- `--all-tools` flag or config key to opt out of progressive mode
- Measured: Claude Code context usage drops ≥15k tokens vs. current

---

#10 Lock docs numbers to a single source of truth

Labels: `docs`, `p2`, `size:S` **Parent:** #5

The site and docs currently disagree: 19+ vs 18+ tools, 38+ vs 28+ commands, 10 vs 8 MCP tools. For a *verification* tool, inconsistent numbers are a credibility problem.

Acceptance.

- Numbers generated from code at build time (manifest parsed, not hand-typed)
- Single `docs/stats.json` (or equivalent) read by all docs pages and the marketing site
- CI check fails if hand-typed counts appear in MDX
- README also reads from the manifest

🗺️ Milestone 2 — Week 2: Constitution + PR comment

#11 Epic — Rebuild constitution as derived, not authored

Labels: `epic`, `constitution`, `p0` **Size:** L

Problem. A blank `constitution.md` forces AI to infer rules from the diff it's supposed to verify against the prior. Spec Kit's issue #366 documents the same failure mode. Claude Code, Windsurf, and CodeRabbit have all pivoted to scan + infer + interview.

Goal. Reshape the flow from `init → prompt → store` to `init → scan → infer → propose → confirm → store → learn`.

Acceptance.

- On a fresh mid-sized public repo, `maina learn` produces ≥10 evidence-cited rules in under 30 seconds
- Human reviewer accepts ≥70% of proposed rules unchanged
- Each rule carries `<!-- source: path:line, confidence: 0.XX -->`

- Interview asks ≤5 questions, only for what scan couldn't infer
- Weighting: 70% scan, 20% interview, 10% ongoing learn

Child issues: #12, #13, #14, #15, #16, #17

#12 Lint-config + manifest parsers

Labels: `constitution`, `p0`, `size:M` **Parent:** #11

Emit `(rule_text, confidence, source_file:line)` triples from:

- `package.json`, `pyproject.toml`, `Cargo.toml`, `go.mod`, `composer.json`
- `.eslintrc*`, `eslint.config.*`, `biome.json`
- `ruff.toml`, `pyproject.toml [tool.ruff]`
- `.editorconfig`, `tsconfig.json`, `.prettierrc*`

Acceptance.

- ≥80% of lint rules mapped 1:1 to constitution statements
- Confidence 1.0 when explicit in config, 0.6 when inferred
- Unit tests against 5 real-world public repo configs

#13 Tree-sitter pattern sampler (TS, Python)

Labels: `constitution`, `p0`, `size:M` **Parent:** #11

Sample ≤100 files per language, detect:

- Async style (async/await vs .then)
- Function style (arrow vs declaration)
- Import style (named vs default)
- Test framework + assertion style
- Error-handling pattern

Emit as medium-confidence (0.4–0.7) rules the user confirms.

Acceptance.

- Runs in under 10 seconds on a 1000-file repo
- Go, Rust, Java, C#, PHP stubbed but gated behind flag
- Output deterministic across runs

#14 Git-log + CI analyzer

Labels: `constitution`, `p1`, `size:S` **Parent:** #11

Extract:

- Conventional Commits usage rate
- Hot paths (top 20 churn files)
- Required CI checks from `.github/workflows/*.yml`
- Branch protection hints from `CODEOWNERS`

Acceptance.

- Runs in under 5 seconds on repos up to 10k commits
- Emits 3–5 constitution rules about workflow conventions
- Graceful degradation on shallow clones

#15 Import adapters for existing rule files

Labels: `constitution`, `p0`, `size:S` **Parent:** #11

Read (don't duplicate) existing:

- `CLAUDE.md`, `AGENTS.md`
- `.cursor/rules/*.mdc`, `.cursorrules`
- `.windsurfrules`, `.windsurf/rules/*`
- `.github/copilot-instructions.md`
- `CONTRIBUTING.md` (section-filtered)

Merge into `constitution.md` with provenance comments.

Acceptance.

- No duplication: if a rule exists in `CLAUDE.md`, it's linked not copied
- Drift detection: flag if imported rule contradicts scan-inferred rule
- Frontmatter `imported_from:` on each imported block

#16 Interview gap-filler + interactive confirm

Labels: `constitution`, `p1`, `size:M` **Parent:** #11

After scan, ask 3 fixed questions:

1. Files AI should never touch? (glob)
2. Deploy-time gotchas?
3. What does every new contributor get wrong?

Then present rules in an interactive accept/reject UI (like `git rebase -i`): `y/n/e` per rule.

Acceptance.

- Non-TTY mode writes draft without interaction
- Rejected rules persist in `.maina/rejected.yml` so subsequent scans don't re-propose
- `maina learn --update` flags new proposals only

#17 Glob-scoped constitution rules via `constitution.d/*.md`

Labels: `constitution`, `p1`, `size:S` **Parent:** #11

Support monorepos and mixed-stack repos with per-path rule files.

```
constitution.md                # root rules
constitution.d/
  frontend.md                  # applies_to: ["apps/web/**"]
  api.md                       # applies_to: ["apps/api/**"]
  mobile.md                    # applies_to: ["apps/mobile/**"]
```

Acceptance.

- Loader merges root + `constitution.d/*.md` in deterministic order
- `applies_to:` globs filter rules per file path

- `maina learn` can propose into the correct sharded file
- Cookbook: “Monorepo constitutions”
- Backward compatible: absent `constitution.d/` → behaves as today

#18 Epic — PR comment v2 (sticky + self-hosted + Checks)

Labels: `epic`, `pr-comment`, `verify`, `p0` **Size:** L

Problem. The existing GitHub Action posts inline review comments on findings — keep that. What’s missing: a **root sticky comment** summarizing the verdict, a **self-hosted permalink** to the full report with screenshots, and a **Checks API entry** so required-check rules can gate merges. Today there’s no link to the verification JSON or screenshots in the PR.

Goal. One sticky root comment (edited in place), one Check Run, one self-hosted permalink — layered on top of the existing inline comments.

Acceptance.

- Exactly one root Maina comment per PR (edited in place via `<!-- maina:run -->` marker)
- Inline screenshots via `mainahq.com/r/<run-id>/shot/*.png`
- `mainahq.com/r/<run-id>` permalink resolves with JSON + HTML view
- GitHub Check Run present in PR status box
- Slash commands `/maina retry`, `/maina explain`, `/maina approve` work
- Existing inline review comments behavior preserved

Child issues: #19, #20, #21, #22, #23

#19 Sticky root-comment writer

Labels: `pr-comment`, `p0`, `size:S` **Parent:** #18

Find-or-create: search PR comments for `<!-- maina:run -->` marker, edit if found, create if not.

Comment template.

```
<!-- maina:run id=abc123 -->
```

```
## 🛡️ Maina verification — ✅ passed (18/20)

> Run `abc123` · commit `a1b2c3d` · 37s · https://mainahq.com/r/abc123

|   | Check | Result |
|---|---|---|
| ✅ | Login flow renders | pass |
| ✅ | Checkout E2E | pass |
| ⚠️ | A11y contrast on /pricing | 2 warnings |
| ❌ | Visual diff: Header.tsx | 1 pixel drift |

### 🖼️ Visual changes (1)
**components/Header.tsx** — `/` at 1440×900

| Before | After |
|---|---|
| ![before](https://mainahq.com/r/abc123/shot/header-before.png) | ![after](https://

<details><summary>📋 Full check results (20)</summary>…</details>
<details><summary>⚠️ Warnings (2)</summary>…</details>

---
📄 Full report: mainahq.com/r/abc123 · 📦 JSON: mainahq.com/r/abc123.json · 🔁 `
```

Acceptance.

- Race-safe (two CI runs in flight → still one root comment)
- Edit preserves comment id so reactions persist
- Graceful handling when `issues:write` perm is missing

#20 Self-hosted report viewer at `mainahq.com/r/<run-id>`

Labels: `pr-comment`, `infra`, `p0`, `size:M` **Parent:** #18 **Depends:** #1

- `POST /api/runs` from CI → returns run-id + signed upload URLs
- `GET /r/<run-id>` → HTML report
- `GET /r/<run-id>.json` → raw JSON
- `GET /r/<run-id>/shot/<id>.png` → screenshot

Storage: R2 (existing).

Acceptance.

- 90-day retention configurable per plan
- Purge endpoint for GDPR-style deletion
- CDN cache headers set
- Robots: noindex on run pages

#21 GitHub Checks API integration

Labels: `pr-comment` , `p0` , `size:M` **Parent:** #18

Emit Check Run “Maina verification”:

- Conclusion: success / failure / neutral
- Summary: “18/20 passed · 2 warnings”
- Details URL: `mainahq.com/r/<run-id>`
- Up to 50 line annotations for failing assertions
- Actions: `[Re-run]` → `workflow_dispatch`

Acceptance.

- Works under both `GITHUB_TOKEN` and GitHub App auth
- Annotations link to exact file:line in the PR diff
- Required-check rule can gate merges

#22 Screenshot pipeline → report bucket

Labels: `pr-comment` , `verify` , `p1` , `size:S` **Parent:** #18

When the Verify engine captures a screenshot, upload to `mainahq.com/r/<run-id>/shot/<id>.png` . Before/after pairs for visual-diff checks.

Acceptance.

- Upload synchronous with run completion
- PNGs optimized (pngquant) to ≤150KB typical
- Alt text includes viewport + route

#23 Slash commands `/maina retry` `/maina explain` `/maina approve`

Labels: `pr-comment`, `p1`, `size:S` **Parent:** #18

Listen for `issue_comment` webhook, parse `/maina <cmd>`, emit `repository_dispatch`:

- `retry` → re-run the CI workflow
- `explain` → post a reply with per-check reasoning
- `approve` → mark a warning as accepted, persist to `constitution.md`

Acceptance.

- ACL: PR author + repo write-perm users only
- Bot reacts with 👀 on receipt, ✅ on completion
- `/maina approve` opens a follow-up PR editing `constitution.md`

🗺️ Milestone 3 — Week 3: Swaps + error reporting

#24 Swap custom graph viz → `react-force-graph-2d`

Labels: `wiki`, `kill`, `p0`, `size:S`

Delete the custom renderer. Wire `react-force-graph-2d` (MIT, ~150KB) to the existing `{nodes, links}` dump. Keep zoom/pan/highlight parity.

Acceptance.

- 60fps at 500 nodes, usable at 2k
- Click → navigate to wiki page
- Hover → metadata tooltip
- ≥500 LOC of custom rendering deleted

#25 Add Pagefind to docs site

Labels: `docs`, `search`, `p0`, `size:XS`

`npx pagefind --site public` in CI. Ship default UI behind ⌘K.

Acceptance.

- Search index ≤400KB for current doc set
- Typo tolerance works (“instal” → “install”)
- ⌘K / Ctrl+K opens modal

#26 Kill decision — @workkit for wiki search

Labels: `adr` , `kill` , `search` , `size:XS`

Close with: no evidence @workkit is a search library. Using Pagefind for docs, Orama for in-app. Revisit if @workkit ships search features.

Acceptance.

- `docs/decisions/0006-no-workkit-search.md` written
- Linked from internal roadmap references

#27 Wire Argos or Lost Pixel into Verify engine

Labels: `verify` , `p0` , `size:M`

Pick one (Argos for managed free-OSS plan; Lost Pixel for fully self-hosted). Integration: baseline capture, PR diff, GitHub Check.

Acceptance.

- `docs/decisions/0007-visual-diff-backend.md` comparing the two
- Replaces any bespoke PNG diff code
- Baselines stored and retrieved per-branch
- Report links from the sticky PR comment (#19)

#28 Kill decision — Passmark adoption

Labels: `adr` , `kill` , `verify` , `size:XS`

Close with: Passmark is not a Playwright replacement — it wraps Playwright. Stagehand does the same under MIT with a larger community and no Redis requirement. FSL license has a “Competing Use” clause. 185 stars is not battle-tested for a verification tool.

Acceptance.

- `docs/decisions/0008-no-passmark.md` written
- Stagehand evaluation scheduled for #44

#29 Kill decision — Fern + multi-language SDKs

Labels: `adr` , `kill` , `docs` , `size:XS`

Close with: No REST API to wrap. MCP is the polyglot surface. Peers (Ruff, uv, Biome, Bun, Vitest, Astro) ship zero SDKs. $600/mo × N languages is not justified. Revisit 2027 Q2 if hosted REST API exists — use Speakeasy then, not Fern.

Acceptance.

- `docs/decisions/0009-no-fern-no-sdk.md` written
- Blog post drafted: "Why Maina has no SDK" (ships in #50)
- Roadmap tickets referencing Fern/SDK closed as won't-fix with link

---

#30 Ship Maina as a Playwright MCP interop partner

Labels: `verify` , `mcp` , `marketing` , `p1` , `size:S`

Publish "Maina works with Playwright MCP" (`@playwright/mcp` , Microsoft, Apache-2.0). Free distribution win via Microsoft's MCP directory and marketing.

Acceptance.

- Verify engine can optionally drive via Playwright MCP
- Docs page: "Using Maina with Playwright MCP"
- Example: Claude Code agent → Playwright MCP → Maina verifier → report
- Submitted to modelcontextprotocol.io / relevant MCP registry
- Launch tweet/blog drafted

#31 Reposition wiki as "a view of the Context engine"

Labels: `marketing` , `wiki` , `positioning` , `p1` , `size:S`

The wiki is a *view*, the Context engine is the product. Current messaging treats the wiki as a separate product surface. Fix across README, site copy, docs IA, and internal vocabulary.

Acceptance.

- README rewritten: wiki under “Context engine → views”
- mainahq.com nav + docs sidebar reflect this IA
- Glossary: Context engine (core) → wiki (view), blast radius (view), symbol graph (view)
- `docs/decisions/0010-wiki-is-a-view.md`

#32 Epic — Automatic error reporting for OSS and Cloud

Labels: `epic`, `observability`, `dx`, `cloud`, `oss`, `p0` **Size:** L **Depends:** #4

Problem. Without error reporting, bugs in the big Week 2 and Week 4 feature ships fly blind. Users hit crashes, open vague “it didn’t work” issues, maintainers can’t reproduce. Cloud users have zero debugging path when something fails server-side.

Design principles.

- **OSS: opt-in, default off.** Single-keypress prompt at install. Aggressive scrubbing. Zero PII.
- **Cloud: opt-out, default on.** Account-linked. Still scrubbed for code content.
- **Separate from usage telemetry (#47).** Two independent toggles.
- **User-visible error IDs.** Every error surfaces an ID the user pastes into a GitHub issue for instant correlation.

Acceptance.

- OSS: opt-in toggle, 0% data capture until consent
- Cloud: opt-out toggle, errors linked to account
- PII and code content scrubbed (automated test + manual audit)
- Error ID visible in CLI stderr, MCP error responses, PR comments on verify failure
- Source maps uploaded in CI so stack traces are readable
- Release tracking enabled so errors bucket by Maina version
- Rate limiting + retention policy documented

Child issues: #33, #34, #35, #36, #37

#33 PII and code-content scrubbing library

Labels: `observability`, `security`, `p0`, `size:M` **Parent:** #32

Load-bearing. Both OSS and Cloud paths depend on this. Runs on the client before network send.

Must redact.

- Absolute file paths → repo-relative
- File contents / code snippets in stack frames → `[redacted]`
- Environment variable values → key names only
- Usernames, emails, IP addresses
- API keys, tokens (broad regex patterns)
- Repo names → SHA-256 hash
- Commit messages, branch names

Must keep.

- Error class, message (after regex scrub)
- Stack trace structure, line numbers, function names
- OS, Node/Python version, Maina version
- Anonymized agent identifier (claude / cursor / copilot / windsurf / zed / none)
- Command invoked (subcommand only, not full argv)

Acceptance.

- 30+ adversarial unit tests (leaked keys, user paths, code snippets)
- Fuzz test: 1000 random inputs, zero leaked PII on output
- Audit checklist in `docs/security/error-scrubbing.md`
- Runs on client, never server-side

#34 OSS error reporting — opt-in, aggressive scrubbing

Labels: `observability`, `oss`, `p0`, `size:M` **Parent:** #32 **Depends:** #4, #33

Install-time prompt: "Share anonymous crash reports to help fix bugs? [y/N]". Decision stored in `~/.maina/config.yml`. `maina errors off` permanently disables.

Scope: uncaught exceptions in the Maina CLI, MCP server panics, Verify engine crashes.

Acceptance.

- Zero events sent until explicit consent
- `maina errors status` shows state + endpoint + last-sent timestamp
- `maina errors off` disables and clears any queued events
- DSN baked into binary, no runtime config needed
- Events include release tag for bucketing
- Consent prompt appears at first run only
- Tested on Linux, macOS, Windows

#35 Cloud error reporting — opt-out, account-linked

Labels: `observability`, `cloud`, `p0`, `size:M` **Parent:** #32 **Depends:** #4, #33

Default-on for Cloud users. Events tagged with user_id + org_id + plan_tier so support can triage.

Acceptance.

- Cloud dashboard shows per-user error history for paid plans
- User-settings toggle to opt out (dashboard + API)
- Opt-out is immediate, clears in-flight events
- Events include user_id, org_id, plan_tier — never email or name
- Support runbook: "customer reports bug → find event in backend" in ≤30s
- GDPR delete: `DELETE /api/user/<id>` purges all events

#36 Error ID surface (CLI, MCP, PR comments)

Labels: `observability`, `dx`, `pr-comment`, `p0`, `size:S` **Parent:** #32

Every captured error surfaces a short ID the user can quote in a GitHub issue.

Surfaces.

- CLI stderr: `Error ERR-ab12cd. Report at github.com/mainahq/maina/issues (include this ID).`
- MCP tool error responses: `{ "error": "...", "error_id": "ERR-ab12cd" }`
- PR comment on verify failure: ⚠️ `Verification crashed. Error ID: ERR-ab12cd.`
- `maina doctor`: shows last 5 error IDs

Acceptance.

- IDs are 6–8 chars, no ambiguous chars (O/0, I/l)
- ID maps 1:1 to an event for maintainer lookup
- GitHub issue template has a required error-ID field
- Maintainer workflow documented: ID → backend → stack trace in ≤10s

---

#37 Source map + release tracking in CI

Labels: `observability`, `ci`, `p1`, `size:S` **Parent:** #32

Without source maps, stack traces are unreadable minified garbage.

Acceptance.

- GitHub Actions step uploads source maps on tag push
- Release version tag in every event matches `package.json` / `pyproject.toml`
- Stack traces link back to GitHub source at the exact commit
- Covers `@mainahq/cli`, MCP server, any TS/Node modules
- Python components have equivalent `sentry-sdk` pipeline

🗺️ Milestone 4 — Week 4: Moat layer begins

#38 Epic — SCIP ingest for TypeScript

Labels: `epic` , `moat` , `wiki` , `p0` **Size:** L

Problem. Custom parsers don't scale and don't interop. SCIP (Apache-2.0, Sourcegraph) is the successor to LSIF — 5–10× smaller, 3× faster. Gives Maina precise code intelligence for free.

Goal. Every TS symbol becomes a wiki node with file, kind, refs, and LLM-generated prose.

Acceptance.

- `scip-typescript` runs in the ingest path
- Emits `wiki/symbols/<fully-qualified-name>.md` with frontmatter
- Symbol pages render in the wiki UI
- Refs graph navigable (click symbol → callers/callees)

Child issues: #39, #40, #41

#39 Run scip-typescript in ingest

Labels: `moat` , `wiki` , `p0` , `size:M` **Parent:** #38

Subprocess invocation, parse protobuf, normalize into Maina's internal node/edge schema.

Acceptance.

- Handles monorepos with multiple `tsconfig.json`
- Incremental reindex on single-file change
- ≤30s on a 10k-file TS repo

#40 Symbol page templates

Labels: `moat` , `wiki` , `p1` , `size:S` **Parent:** #38

Per-symbol markdown with:

- Signature block
- File + line
- Refs (inbound + outbound)
- Auto-generated prose via one LLM pass

- Mermaid diagram for call relationships

Acceptance.

- LLM prose has content-hash-keyed deterministic caching
- Diagrams render without JS in markdown view
- Page loads in under 100ms cold

#41 DeepWiki-compatible MCP server

Labels: `moat`, `mcp`, `p0`, `size:M` **Parent:** #38

Expose:

- `ask_question(repo, question)`
- `read_wiki_structure(repo)`
- `read_wiki_contents(repo, page)`

Matches DeepWiki's surface exactly → instant interop with any client that already speaks DeepWiki.

Acceptance.

- All 3 tools pass DeepWiki's test harness (or equivalent)
- Published as `maina-wiki-mcp` so users can run standalone
- Documented under "MCP Tools"

#42 LSIF → SCIP migration audit (conditional)

Labels: `moat`, `wiki`, `audit`, `p2`, `size:XS`

Sourcegraph migrated off LSIF in v4.6. If Maina currently uses LSIF anywhere, migrate to SCIP as part of #38. If not, close as N/A.

Acceptance.

- `rg -i "lsif"` across the codebase
- If hits: migration plan written, folded into #38
- If no hits: `docs/decisions/0011-no-lsif.md` closes it out

🗺️ Milestone 5 — Week 5: Differentiator + experiments

#43 Blast-radius view using stack-graphs

Labels: `moat` , `verify` , `wiki` , `p0` , `size:L`

Use `github/stack-graphs` (MIT, file-local, no build required) to compute "if this symbol changes, what else is affected?" Tie into Verify engine: diffs touching high-blast-radius symbols get escalated verification.

This is the single clearest differentiator against DeepWiki and peers.

Acceptance.

- Blast-radius panel in the wiki for every symbol
- Verify engine reads radius on PR diff, adjusts check depth
- Documented as Maina's wedge — "the thing DeepWiki doesn't do"
- Loom demo comparing Maina vs DeepWiki on identical public repo

---

#44 Stagehand spike (feature-flagged)

Labels: `verify` , `experiment` , `p1` , `size:M` **Depends:** #2

Parallel implementation of one Verify flow in scripted Playwright vs Stagehand. Measure across 50 runs: reliability, token cost, time-to-first-green, cache hit rate after first run.

Acceptance.

- Feature flag `verify.ai_flow_discovery=true`
- Benchmark results in `docs/experiments/stagehand-vs-playwright.md`
- Go/no-go decision at end of week against criteria in #2
- If go: documented cost ceiling + cache policy

#45 Orama in-app hybrid search prototype

Labels: `wiki` , `search` , `experiment` , `p1` , `size:M` **Depends:** #2

Reuse the Context engine's embeddings. Compare hybrid (BM25 + vector) vs lexical-only

on 100 wiki pages. Metric: NDCG@10 on a 30-query test set.

Acceptance.

- Benchmark methodology documented
- Results in `docs/experiments/orama-hybrid-search.md`
- Ship only if the gates in #2 are met

🗺️ Milestone 6 — Week 6: Polish + instrumentation

#46 Cookbook pages

Labels: `docs`, `p1`, `size:S`

Minimum four:

1. Verify a PR in CI
2. Let Claude Code self-check before committing
3. Integrate Maina with CodeRabbit workflow
4. Enforce `constitution.md` as a required check

Acceptance.

- Each has a runnable example repo linked
- Navigable via docs sidebar "Cookbooks"

#47 Opt-in usage telemetry (distinct from error reporting)

Labels: `dx`, `instrumentation`, `p1`, `size:M`

Single-keypress at install: "Share anonymous usage? [y/N]". Separate from crash-report consent. PostHog OSS or trivial event POSTs.

Events (all anonymous):

- `maina.install`
- `maina.verify.started` / `.completed`
- `maina.learn.ran`

- `maina.error` (error class only, no stack)

Acceptance.

- Schema in `docs/telemetry.md`
- `maina telemetry off` permanently disables
- Zero PII verified via code review + audit checklist
- Telemetry consent prompt is distinct from error-reporting consent

---

#48 `llms.txt` and `llms-full.txt`

Labels: `docs`, `p1`, `size:XS`

Add `starlight-llms-txt` plugin to the docs site.

Acceptance.

- `docs.mainahq.com/llms.txt` resolves
- Content covers: pitch, install, MCP tools, cookbooks index
- Linked from `<head>` via `<link rel="llms">`

#49 Deprecation sweep

Labels: `kill`, `cleanup`, `p2`, `size:S`

Close-as-won't-fix any open ticket referencing: Fern, SDK generation, custom graph renderer, custom search, LSIF (if #42 confirmed no hits), Passmark, @workkit-as-search.

Acceptance.

- All closed tickets have a comment with the ADR link
- Roadmap doc updated
- README architecture section updated

#50 Kill-post blog series

Labels: `marketing`, `kill`, `p2`, `size:S`

OSS community respects killed scope. Four posts ≤800 words each:

1. “Why Maina has no SDK” (pairs with #29)
2. “Why we’re not adopting Passmark” (pairs with #28)
3. “Why we’re not building our own search” (pairs with #25, #26)
4. “Why the wiki is a view, not a product” (pairs with #31)

Acceptance.

- Each links to its corresponding ADR
- Published to mainahq.com/blog or equivalent
- Shared in at least one developer community (HN, Lobsters, relevant Discord)

🧭 Dependency graph

```
Milestone 0 (pre-sprint ADRs):
        #1, #2, #3, #4

Week 1: #5 (→ #6, #7, #8, #9, #10)

Week 2: #11 (→ #12, #13, #14, #15, #16, #17)
        #18 (→ #19, #20, #21, #22, #23)
        #20 depends #1

Week 3: #24, #25, #26, #27, #28, #29, #30, #31
        #32 (→ #33, #34, #35, #36, #37)
        #34, #35 depend #4, #33

Week 4: #38 (→ #39, #40, #41), #42

Week 5: #43 depends #38
        #44 gated by #2
        #45 gated by #2

Week 6: #46, #47, #48, #49, #50
```

🚦 Recommended P0 order if only 5 ship

1. **#6** — Split `/getting-started` into `/quickstart` + `/full-setup` (the actual onboarding fix)
2. **#11** — Constitution rebuild (scan first, author second)
3. **#18** — PR comment v2 (sticky + Checks + self-hosted report)
4. **#32** — Automatic error reporting (OSS + Cloud)
5. **#43** — Blast-radius view using stack-graphs (the wedge)

Everything else is accelerant.

📊 Summary

- **50 issues total** (7 epics, 39 tasks, 4 pre-sprint ADRs)
- **11 ADRs** covering storage, experiment gates, CLI/MCP co-equality, error backend, visual-diff backend, kill decisions for @workkit / Passmark / Fern / SDK, no-LSIF, wiki-as-view
- **Milestone 0 should land before Week 1 starts** — ~1 day of written decisions that unblock downstream work
- **Size budget:** ~5 L, ~14 M, ~13 S, ~13 XS. One engineer full-time → ~60% of this in 6 weeks; two engineers → all of it with room for surprises.